

TÀI LIỆU KỸ THUẬT

FairDispatch -- MOMAQL

Hướng dẫn Kỹ thuật & Khả năng Tái lập -- tài liệu đồng hành với Research Report; tài liệu này trình bày phần triển khai, không tranh luận lại claim khoa học.

Git commit tại thời điểm build: 3950e7fed5a5

Cấu trúc mã nguồn bên dưới khớp cả với 03_Source_Code_Va_Ket_Qua/ của gói nộp này lẫn repo dev đã copy ra (fairdispatch_v3_clean/) -- cùng đường dẫn tương đối, chỉ khác tên thư mục gốc.

MỤC LỤC

1. Tổng quan Hệ thống (System Overview).....	4
2. Cấu trúc Repository (Repository Structure).....	4
3. Môi trường (Environment).....	5
4. Hợp đồng Dữ liệu (Data Contract).....	6
4.1 Schema parquet gốc (train/val/test).....	6
4.2 Request dict đã xử lý (in-memory, mỗi dòng).....	6
4.3 Trạng thái Driver (src/simulator.py:Driver).....	7
4.4 State/Action/Reward RL (MOMAQLPolicy).....	7
5. Đặc tả Module (Module Specification).....	7
5.1 src/simulator.py	7
5.2 src/policies.py.....	8
5.3 common_loader.py.....	8
6. Thuật toán / Pseudocode (Algorithm).....	9
6.1 Vòng lặp dispatch theo lô (run_simulation_batched).....	9
6.2 Score MOMAQL và cập nhật Q.....	9
7. Cấu hình (Configuration)	9
8. Protocol & Khả năng Tái lập Final Test (Final Test Protocol & Reproducibility).....	10
8.1 Protocol Đóng băng (Frozen Protocol).....	10
8.2 Data Quality Transform (raw file bất biến, tại thời điểm đánh giá).....	11
8.3 Lệnh chạy Final Test.....	11
8.4 Bản đồ Artifact (final_test/).....	12
8.5 Phụ lục Định nghĩa Metric.....	12
8.6 Giới hạn Khả năng Tái lập (Final Test).....	13
9. Lệnh Tái lập Chính xác (Exact Reproduction Commands).....	13
10. Gói Khả năng Tái lập (Reproducibility Package).....	14
11. Vấn đề Đã biết, Giả định, và Xử lý sự cố (Known Issues, Assumptions, and Troubleshooting).....	15
11.1 Giả định (khớp Research Report Mục 11.3, theo góc nhìn kỹ thuật).....	15
11.2 Vấn đề Đã biết (Known Issues).....	16
11.3 Xử lý sự cố (Troubleshooting).....	16

DANH MỤC HÌNH ẢNH

Hình 1. Hình 1: Chuyển đi NYC TLC 2013 thật -> lọc Manhattan + chất lượng -> 67 zone taxi TLC -> $Q(\text{vùng, giờ})$ huấn luyện qua Bellman TD(0) -> simulator theo lô cửa sổ 60 giây -> ghép cặp Hungarian M-to-N -> chỉ số utility/Gini/variance.....	4
--	---

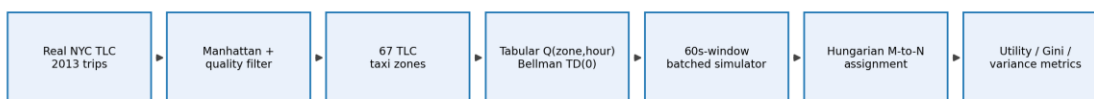
DANH MỤC BẢNG BIỂU

Bảng 1. Môi trường phần cứng/phần mềm đã audit tại thời điểm build tài liệu.....	5
Bảng 2. Vai trò của từng phần chia dữ liệu (train/val/test).	6
Bảng 3. Các hàm chính trong src/simulator.py.....	7
Bảng 4. Công thức score_fn của 5 policy điều phối.	8
Bảng 5. Các hàm trong common_loader.py.	8
Bảng 6. Hằng số cấu hình cấp module (không có central config file).	10
Bảng 7. Protocol đóng băng dùng cho Final Held-out Test.....	11
Bảng 8. Số dòng bị ảnh hưởng qua từng bước data quality transform (test.parquet).	11
Bảng 9. Bản đồ artifact trong final_test/.....	12
Bảng 10. Lệnh tái lập chính xác, theo đúng thứ tự chạy.	14
Bảng 11. Gói khả năng tái lập (reproducibility package) đầy đủ.	15

1. Tổng quan Hệ thống (System Overview)

FairDispatch -- MOMAQL là một bản tái lập độc lập, xây lại từ đầu, các xu hướng định tính của Kang et al. [2024], "Long-term Fairness in Ride-Hailing Platform" (ECML PKDD 2024, arXiv:2407.17839): một chính sách điều phối RL đa mục tiêu (MOMAQL) đánh đổi giữa công bằng thu nhập tài xế và hiệu quả hệ thống, dùng ước lượng giá trị nhìn trước (look-ahead) để tránh quyết định công bằng cận thị từng cửa sổ. Dự án này đánh giá ý tưởng đó trên dữ liệu taxi NYC TLC 2013 thật (912.375 huấn luyện / 195.508 kiểm định / 195.510 kiểm thử), so với 4 chính sách baseline, rồi xác minh lại mọi phát hiện trên một tập Test tách riêng theo thời gian mà implementation chưa từng thấy trong lúc phát triển (Mục 8). Tài liệu này là tài liệu đồng hành về kỹ thuật/khả năng tái lập -- không tranh luận lại claim nào được tái lập; verdict đó ("Strong Partial Trend Replication with held-out temporal support") cùng lý giải đầy đủ theo từng claim nằm ở Research Report (docs/docx_report/).

Pipeline đầu-cuối thật được thực thi bởi repo này:



Hình 1. Hình 1: Chuyển đi NYC TLC 2013 thật -> lọc Manhattan + chất lượng -> 67 zone taxi TLC -> Q(vùng,giờ) huấn luyện qua Bellman TD(0) -> simulator theo lô cửa sổ 60 giây -> ghép cặp Hungarian M-to-N -> chỉ số utility/Gini/variance.

Không có message queue, không database, không web service -- mọi giai đoạn là một script Python thuần đọc/ghi file cục bộ. Simulator là một vòng lặp discrete-event (DES) nhẹ, chạy trong cùng process (src/simulator.py), không phải hệ thống real-time.

2. Cấu trúc Repository (Repository Structure)

```
03_Source_Code_Va_Ket_Qua/ (thư mục gốc của gói nộp này -- cấu trúc tương đối
|                             giống hệt repo dev fairdispatch_v3_clean/ đã copy ra,
|                             chỉ khác tên thư mục gốc)
|-- src/
|   |-- simulator.py          dataclass Driver/SimResult, init_drivers,
|   |                         feasible_drivers, commit_trip, run_simulation_batched,
|   |                         run_simulation_with_horizon, run_simulation (cũ)
|   |-- policies.py           hungarian_batch_assign + 5 class policy (Greedy,
|   |                         Nearest, LAF, ExactReassignPolicy, MOMAQLPolicy)
|-- common_loader.py          loader parquet -> dict request; gini/variance/std/CV
|-- train_momaql.py           huấn luyện bằng Q(vùng,giờ) canonical (1 lượt)
|-- train_momaql_multipass.py ĐÃ CÔNG BỐ KHÔNG ỔN ĐỊNH -- không dùng để đánh giá
|-- run_r1.py                 R1: so sánh baseline 5 policy
|-- run_r2_ablation.py        R2: Full / w/o-Forecast / w/o-Fairness
|-- run_pareto_frontier.py    quét lambda, 7 giá trị
```

```

|-- run_multi_horizon.py      quỹ đạo multi-horizon + tỷ lệ đổi quyết định policy
|-- run_complete_verifications.py  quét quy mô đội xe + đổi quyết định theo không gian
|-- run_spatial_candidate_pool.py  độ sâu vùng ứng viên, lỗi vs. ngoại vi
|-- run_q_table_convergence.py    hội tụ bảng Q theo ngày (37 ngày)
|-- run_hypothesis1_weekly_cycle.py  kiểm tra cơ chế chu kỳ nhu cầu tuần
|-- run_hypothesis4_fairness_balance.py  trace cân bằng score fairness/nhìn trước
|-- train_and_eval_mlp.py        MLP PyTorch thật dự báo nhu cầu vs. Q dạng bảng
|-- make_report_figures.py       sinh lại mọi hình từ reports/*.csv
|-- tests/
|   `-- test_simulator_invariants.py  20 test bất biến (cần có data/*.parquet
|                                       cục bộ để chạy -- Mục 11.3)
|-- scripts/final_test/          Pipeline Final Held-out Test (Mục 8): quality
|                               transform, script chạy baseline/ablation/long-horizon,
|                               script tổng hợp claim assessment
|-- data/                       các phần chia train/val/test parquet (gitignore, xem
|                               Mục 10 để lấy SHA-256) + JSON bảng Q đã huấn luyện
|-- reports/                    mọi CSV/JSON/PNG Validation được giữ lại, trích dẫn ở cả 2 tài
liệu
|-- final_test/                 Kết quả Final Held-out Test (Mục 8): protocol,
|                               data quality gate, kết quả từng seed, claim
|                               assessment, mentor summary, hình, log
`-- docs/
    |-- docx_report/            Research Report (.docx) + hình của nó
    |-- techdoc/               tài liệu này
    `-- ride_hailing_fairness_report_en(vi)/  paper LaTeX + PDF đã compile

```

Repo này không có dependency lockfile, file package-metadata, Dockerfile, CI workflow, hay central config file nào. Mỗi `run_*.py` là một script độc lập với hằng số riêng ở cấp module (`N_DRIVERS`, `SEEDS`, v.v.) -- xem Mục 7.

3. Môi trường (Environment)

Môi trường đã audit tại thời điểm build (đây là máy repo này được verify lần cuối -- KHÔNG tuyên bố mọi thí nghiệm lịch sử đều chạy trên phần cứng giống hệt; không giữ lại environment lockfile từ các lần chạy trước).

Thành phần	Giá trị
OS	Windows 11 (10.0.26200)
Python	3.12.10
CPU	Intel Core i5-10300H, 4 lõi / 8 luồng
RAM	Tổng 15,8 GiB
GPU	NVIDIA GeForce GTX 1650, ~4,3 GB VRAM (có CUDA)
Thư viện chính	numpy 2.2.6, pandas 2.3.2, pyarrow 21.0.0, scipy 1.16.2, matplotlib 3.10.6, pytest 8.3.3, python-docx 1.2.0, torch 2.7.1+cu118

Bảng 1. Môi trường phần cứng/phần mềm đã audit tại thời điểm build tài liệu.

```
python -m venv .venv
.venv\Scripts\activate # Windows; dùng source .venv/bin/activate trên
Linux/Mac
pip install numpy pandas pyarrow scipy matplotlib pytest python-docx
pip install torch # chỉ cần cho train_and_eval_mlp.py
```

Repo này không giữ CI/CD, không Docker image, không requirements.txt -- danh sách package ở trên là bộ đã audit, đang hoạt động; hãy ghim đúng version nếu cần tái lập bit-for-bit qua nhiều máy.

4. Hợp đồng Dữ liệu (Data Contract)

Phần chia	Vai trò
train.parquet	Huấn luyện Q(vùng,giờ) (train_momaql.py); huấn luyện MLP cho sensitivity forecast
val.parquet	Phát triển / phân tích: build+debug simulator, so sánh policy, ablation, quét Pareto, long-horizon, thí nghiệm cơ chế, chốt cấu hình cuối
test.parquet	CHỈ dùng để xác minh held-out cuối cùng theo thời gian (Mục 8) -- không bao giờ dùng để chọn lambda/gamma/alpha, không bao giờ dùng bởi product demo trực tiếp

Bảng 2. Vai trò của từng phần chia dữ liệu (train/val/test).

4.1 Schema parquet gốc (train/val/test)

Các cột được common_loader.load_requests_fast() và loader riêng của train_momaql.py đọc: pickup_ts (timestamp), pickup_latitude, pickup_longitude, dropoff_latitude, dropoff_longitude, fare_amount, duration_seconds, pickup_zone_id, dropoff_zone_id. pickup_ts là epoch time tuyệt đối -- giờ-trong-ngày được tính trực tiếp từ đó (KHÔNG phải offset tương đối theo từng file), nên một khung giờ mang cùng ý nghĩa thời gian thực trong train.parquet lẫn val/test.parquet.

4.2 Request dict đã xử lý (in-memory, mỗi dòng)

```
{
  "_idx": int, # vị trí trong danh sách request đã sắp xếp
  "pickup_ts": float, # giây, TƯƠNG ĐỐI so với dòng đầu tiên của split này
  "pickup_latitude": float, "pickup_longitude": float,
  "dropoff_latitude": float, "dropoff_longitude": float,
  "fare_amount": float, # cước phí USD thật
  "duration_seconds": float,
  "pickup_zone_id": int, "dropoff_zone_id": int, # id zone taxi TLC
  "pickup_hour": int, # 0-23, tính từ epoch time TUYỆT ĐỐI
  "dropoff_hour": int, # 0-23, = (pickup_epoch + duration) % 24h
}
```

4.3 Trạng thái Driver (src/simulator.py:Driver)

```
Driver:
    driver_id: int
    lat, lon: float           # vị trí hiện tại
    available_at: float       # thời điểm sớm nhất driver này nhận chuyến mới
    total_income: float = 0.0 # net tích lũy (fare - deadhead_cost)
    total_deadhead_cost: float = 0.0
    total_trips: int = 0
```

4.4 State/Action/Reward RL (MOMAQLPolicy)

- State S = (pickup_zone_id, pickup_hour) của chuyến driver vừa nhận.
- State S' = (dropoff_zone_id, dropoff_hour) của chính chuyến đó -- driver kết thúc ở đâu/khi nào.
- Action = một phép ghép cặp joint M-to-N cho 1 cửa sổ 60 giây, giải 1 lần mỗi cửa sổ bằng thuật toán Hungarian (không phải chọn greedy từng request).
- Reward = fare_amount - eta_seconds × 0,0025 (hệ số chi phí deadhead, USD mỗi giây ETA đón khách).
- Key bảng Q: (zone_id: int, hour_of_day: int) -> float. Khi serialize JSON dùng key dạng chuỗi "zone:hour" (vd "137:0"); MOMAQLPolicy._parse_q_table() nhận cả tuple key gốc lẫn định dạng chuỗi này.

5. Đặc tả Module (Module Specification)

5.1 src/simulator.py

Hàm	Đầu vào	Đầu ra
init_drivers(n, requests, seed)	số lượng driver, danh sách request, seed RNG	list[Driver], đặt tại n vị trí đón khách thật lấy mẫu từ danh sách request
feasible_drivers(drivers, req, now)	danh sách driver, 1 request, thời điểm hiện tại	list (driver, deadhead_miles, eta_seconds) cho driver rảnh tính tới `now` và trong ngưỡng MAX_PICKUP_ETA_SECONDS=600s
commit_trip(d, req, dist, eta, now, result, record_trace)	driver đã chọn + request + thời gian	thay đổi trạng thái driver (income, vị trí, available_at) và tổng số của result
run_simulation_batched(requests, n_drivers, policy, seed, window_seconds=60.0)	danh sách request đã sắp xếp + đối tượng policy	SimResult (thu nhập từng driver, số chuyến hoàn thành)
run_simulation_with_horizon(..., checkpoint_days, compare_policy=None, zone_classifier=None)	tương tự + các ngày checkpoint + policy so sánh tùy chọn	(result, dict checkpoints, disagreement_rate) -- Một quỹ đạo duy nhất, chụp snapshot ở mỗi ngày checkpoint, không phải chạy lại độc lập

Bảng 3. Các hàm chính trong src/simulator.py.

Đơn giản hóa có chủ đích (đã công bố trong docstring module): vị trí driver là thật (lat, lon), không phải xấp xỉ theo ô không gian; ETA là khoảng cách haversine với tốc độ hằng số 12 mph, không phải mô hình road-routing thật; hệ số chi phí deadhead (0,0025 USD/giây) tái sử dụng nguyên trạng từ UtilityCoefficients đã đóng băng của dự án cha, không tự nghĩ lại.

5.2 src/policies.py

Policy	score_fn(d, req, dist, eta)	Ghi chú
Greedy	fare_amount	giống nhau giữa mọi candidate của 1 request; joint solve vẫn tối đa số chuyển phục vụ
Nearest	$(600.0 - \text{eta}) \times 0,0025$	định tâm lại để 0 = break-even tại ngưỡng feasibility; $\text{argmax}(\text{score}) == \text{argmin}(\text{eta})$
LAF	$[(\text{mean_income} - \text{d.income}) / \max(\text{mean_income}, 1)] \times \text{fare}$	$\text{argmax}(\text{score}) == \text{argmin}(\text{driver.income})$; heuristic chỉ tối ưu fairness
Exact REASSIGN	fare_amount - eta×0,0025	tối đa hóa net-utility thuần, giải joint M-to-N thật
MOMAQL	$(1-\lambda) \cdot [\text{fare} - \text{eta} \times 0,0025 + \gamma \cdot Q(\text{D_zone}, \text{D_hour})] + \lambda \cdot [(\text{mean_income} - \text{d.income}) / \max(\text{mean_income}, 1)] \cdot \text{fare}$	policy duy nhất có Q-learning online (on_committed cập nhật Q qua Bellman TD(0) trừ khi frozen=True)

Bảng 4. Công thức score_fn của 5 policy điều phối.

Cả 5 policy dùng CHUNG một khung tham chiếu: hungarian_batch_assign() giải một phép ghép joint thật qua scipy.optimize.linear_sum_assignment trên ma trận chi phí $(n_{\text{req}} + n_{\text{drv}}) \times (n_{\text{req}} + n_{\text{drv}})$ đệm dummy, nên mọi request/driver đều có lựa chọn "từ chối"/"rảnh" thật với chi phí 0 -- đây là ghép cặp maximum-WEIGHT (đúng công thức $\sum_v I_{rv} \leq 1$ của paper), không phải maximum-cardinality.

5.3 common_loader.py

Hàm	Mục đích
load_requests_fast(path)	loader parquet -> list[dict] dùng PyArrow; tính pickup_hour/dropoff_hour từ epoch time tuyệt đối
gini(values)	hệ số Gini của danh sách thu nhập driver
variance(values) / std(values)	variance/std của toàn bộ tổng thể -- metric fairness chính của chính paper
coefficient_of_variation(values)	std/mean; chặn NaN khi mean < 1e-9 (không ổn định khi utility trung bình gần 0)

Bảng 5. Các hàm trong common_loader.py.

6. Thuật toán / Pseudocode (Algorithm)

6.1 Vòng lặp dispatch theo lô (run_simulation_batched)

```
mỗi cửa sổ 60 giây, theo đúng thứ tự pickup_ts thật:
    window_reqs = request có pickup_ts trong [window_start, window_start+60s)
    for req in window_reqs:
        candidates[req] = feasible_drivers(drivers, req, window_start)
        # rảnh tính tới window_start VÀ eta <= 600s

    assignments = policy.select_batch(candidates, window_start)
    # -> giải Hungarian joint thật, có lựa chọn từ chối chi phí 0

    for req, (driver, dist, eta) in assignments.items():
        if driver đã dùng trong cửa sổ này: skip # chặn lỗi policy
        commit_trip(driver, req, dist, eta, window_start, result)
        # driver.income += fare - eta*0.0025
        # driver.available_at = window_start + eta + duration_seconds
        policy.on_committed(driver, req, dist, eta, window_start)
        # chỉ MOMAQL: cập nhật Bellman TD(0) online (bỏ qua nếu frozen)
```

6.2 Score MOMAQL và cập nhật Q

```
score(d, req) = (1-lambda) * [fare - eta*0.0025 + gamma * Q(D_zone, D_hour)]
               + lambda * [(mean_income - d.income) / max(mean_income,1)] * fare

Cập nhật Q (Bellman TD(0), lúc commit, chỉ khi không frozen):
P = (pickup_zone_id, pickup_hour) # state driver đang ở
S = (dropoff_zone_id, dropoff_hour) # state driver kết thúc ở
reward = fare - eta*0.0025
Q[P] <- Q[P] + alpha * (reward + gamma * Q[S] - Q[P])
```

lambda=0,5 (mặc định), gamma=0,9, alpha=0,1. ablation='no_forecast' ép số hạng $\gamma * Q(\dots)$ về 0; ablation='no_fairness' ép lambda về 0. frozen=True (dùng cho mọi lần đánh giá) tắt hẳn việc cập nhật Q -- bảng đã huấn luyện từ train_momaql.py chỉ được đọc, không ghi.

7. Cấu hình (Configuration)

Không có central config file nào (không YAML/JSON/.env). Mỗi hằng số dưới đây là 1 biến Python cấp module, lặp lại ở từng script (đã verify thật, không giả định):

Hằng số	Giá trị	Định nghĩa ở đâu
N_DRIVERS	200	đầu mỗi run_*.py và train_momaql.py
SEEDS (thí nghiệm chính)	20260721, 20260722, 20260723,	run_r1.py, run_r2_ablation.py, run_pareto_frontier.py,

	20260724, 20260725	run_multi_horizon.py, train_and_eval_mlp.py
SEEDS (thí nghiệm cơ chế)	20260721, 20260722, 20260723 (3 seed)	run_complete_verifications.py, run_spatial_candidate_pool.py, run_hypothesis4_fairness_balance.py
WINDOW_SECONDS	60,0	tham số mặc định của run_simulation_batched
MAX_PICKUP_ETA_SECONDS	600,0 (10 phút)	hằng số module src/simulator.py
COST_PER_SECOND_DEADHEAD_USD	0,0025	hằng số module src/simulator.py
AVG_SPEED_MPH	12,0	hằng số module src/simulator.py
lambda (trọng số fairness)	mặc định 0,5; quét {0; 0,2; 0,4; 0,5; 0,6; 0,8; 1,0} cho Pareto	tham số constructor MOMAQLPolicy(lam=...)
gamma (discount)	0,9	tham số constructor MOMAQLPolicy(gamma=...)
alpha (learning rate Q)	0,1	tham số constructor MOMAQLPolicy(alpha=...)

Bảng 6. Hằng số cấu hình cấp module (không có central config file).

8. Protocol & Khả năng Tái lập Final Test (Final Test Protocol & Reproducibility)

Chi tiết kỹ thuật đồng hành cho Final Held-out Temporal Test Evaluation (Research Report Mục 9). Mọi thứ ở đây đọc trực tiếp từ `final\_test/` tại thời điểm build tài liệu -- không số nào ở đây gõ tay.

8.1 Protocol Đóng băng (Frozen Protocol)

Tham số	Giá trị
Policy	MOMAQL canonical (lambda=0,5, gamma=0,9, alpha=0,1)
Số tài xế	200
Bộ giải ghép cặp	Hungarian joint assignment (scipy.optimize.linear_sum_assignment)
Bảng Q	data/momaql_q_table_trained.json, đã đóng băng (không học thêm lúc đánh giá)
Seed	20260721, 20260722, 20260723, 20260724, 20260725
Quy tắc không tuning	Không đổi config/hyperparameter/policy/simulator sau khi đã xem bất kỳ kết quả Test nào

Bảng 7. Protocol đóng băng dùng cho Final Held-out Test.

Toàn văn protocol (hash mã nguồn, checksum dataset, môi trường): `final\_test/FINAL\_TEST\_PROTOCOL.md`.

8.2 Data Quality Transform (raw file bất biến, tại thời điểm đánh giá)

test.parquet trên đĩa KHÔNG BAO GIỜ bị sửa. Hai rule độc lập chỉ áp dụng lên evaluation view trong bộ nhớ, theo đúng thứ tự này, không trộn lẫn:

- A. Vệ sinh ranh giới thời gian nghiêm ngặt: loại các dòng có giây epoch pickup_ts trùng với giây epoch pickup_ts lớn nhất của val.parquet (một artifact thật do chia theo row-index -- các pickup thật khác nhau rơi cùng 1 giây tại điểm cắt, không phải dữ liệu trùng lặp) -- loại 3 dòng.
- B. Sửa duration tối thiểu, có tính tất định (deterministic): nếu duration_seconds lưu sẵn hợp lệ ($0 < x \leq 24h$), giữ nguyên. Nếu không, mà dropoff_ts-pickup_ts hợp lệ, sửa duration_seconds_eval từ timestamp (quality_action=REPAIRED_FROM_TIMESTAMPS). Ngược lại loại bỏ (không phục hồi được) -- sửa 32 dòng, loại 1 dòng trong lượt này.

Bước	Số dòng
test.parquet gốc	195510
Loại do ranh giới thời gian	3
Sửa duration từ timestamp	32
Loại do duration không phục hồi được	1
Evaluation View cuối cùng của Test	195506

Bảng 8. Số dòng bị ảnh hưởng qua từng bước data quality transform (test.parquet).

Triển khai: `scripts/final\_test/quality\_transform.py` (load_requests_with_quality_transform()), có self-check ở `scripts/final\_test/test\_quality\_transform.py`. Mỗi request được đánh giá đều giữ lại `duration\_seconds\_raw` và `quality\_action` bên cạnh `duration\_seconds` (có thể đã sửa) để audit được từng dòng. Audit trail đầy đủ (trace ngược nguồn xác nhận lỗi tồn tại ở dữ liệu nguồn thô, không phải do preprocessing của dự án này): `final\_test/DATA\_QUALITY\_GATE.md`.

8.3 Lệnh chạy Final Test

```
All commands run from:
D:\ProjectVSF\FairDispatch_MOMAQL_Fair_Ride_Hailing_Dispatch_Replication\03_Source_Code_Va_Ket_Qua

python scripts/final_test/audit_test_dataset.py
python scripts/final_test/test_quality_transform.py
python scripts/final_test/quality_transform.py
python scripts/final_test/verify_before_run.py
python scripts/final_test/run_final_test_baselines.py > final_test/logs/baseline_run.log 2>&1
python scripts/final_test/run_final_test_ablation.py > final_test/logs/ablation_run.log 2>&1
python scripts/final_test/run_final_test_long_horizon.py >
final_test/logs/long_horizon_run.log 2>&1
python scripts/final_test/build_final_test_summary.py
```

No test-driven tuning occurred between any of these steps. Protocol (FINAL_TEST_PROTOCOL.md) and data-quality transform (DATA_QUALITY_GATE.md) were frozen before run_final_test_baselines.py was ever executed.

8.4 Bản đồ Artifact (final_test/)

Đường dẫn	Mục đích
FINAL_TEST_PROTOCOL.md	Config/seed/hash đã đóng băng, viết trước khi bất kỳ policy nào chạy trên test.parquet
DATA_QUALITY_GATE.md	Audit đầy đủ về anomaly duration, trace root-cause, lý giải rule sửa
test_quality_transform_manifest.json	Số liệu sửa/loại dạng máy đọc được + row ID theo từng split
baseline/	5 policy x 5 seed trên Final Test Evaluation View (CSV từng seed + tổng hợp)
ablation/	Full / No Forecast / No Fairness x 5 seed
long_horizon/	Kết quả checkpoint trên 1 quỹ đạo, ngày 1-37 + tỷ lệ đổi quyết định policy
validation_vs_test.csv	So sánh chiều Validation-vs-Test theo từng phát hiện (heldout_generalization)
test_claim_assessment.csv	Bảng claim C1-C6: heldout_generalization + paper_replication_verdict (2 cột độc lập)
FINAL_TEST_MENTOR_SUMMARY.md	Tóm tắt dễ đọc trả lời 6 câu hỏi generalization cốt lõi
figures/	PNG baseline/ablation/long-horizon sinh từ CSV Final Test
logs/	commands.log, environment.txt, runtimes.csv

Bảng 9. Bản đồ artifact trong final_test/.

8.5 Phụ lục Định nghĩa Metric

- Fairness là một KHÁI NIỆM, không phải 1 metric duy nhất -- Gini và Variance là 2 metric được báo cáo (xem công thức ở Mục 5/6 phía trên); Gini thấp hơn và Variance thấp hơn đều nghĩa là thu nhập driver đồng đều hơn.
- Paired delta: với 1 seed, (metric ở config A) - (metric ở config B), tính theo từng seed rồi lấy trung bình - báo cáo kèm tính nhất quán về dấu (vd "5/5 seed") chứ không chỉ mean.
- Generalization (heldout) theo chiều: một phát hiện quan sát trên Validation có lặp lại ĐÚNG CHIỀU trên Test không? Chỉ tính từ dấu/so sánh, không có ngưỡng độ lớn.
- Paper replication verdict: phát hiện có khớp với tuyên bố định tính của chính arXiv:2407.17839 không? Đây là đánh giá độc lập so với paper, KHÔNG suy ra được từ phép tính generalization -- một phát hiện có thể generalize trong khi claim của paper vẫn Not Reproduced (xem ví dụ C4 đã làm ở Research Report Mục 10).

8.6 Giới hạn Khả năng Tái lập (Final Test)

- Dataset là NYC TLC 2013, không phải dữ liệu 2016 gốc của paper -- Final Test kế thừa đúng phạm vi trend-replication như Validation (Mục 3 của Research Report), không phải exact reproduction.
- Chỉ 5 seed -- effect size, mean/std, và tính nhất quán dấu theo seed là bằng chứng chính; không tính hay tuyên bố formal statistical significance test nào.
- Chỉ chạy đúng 1 operating point canonical $\lambda=0,5$ -- không quét λ trên Test (có chủ đích; Test dùng để verify, không dùng để chọn operating point mới).
- Product demo (05_SanPham_Demo) mặc định dùng slice Validation/demo, không bao giờ dùng test.parquet -- Test chỉ dành riêng cho đánh giá khoa học cuối cùng này, không lộ ra trong Control Room trực tiếp.

Môi trường chạy Final Test (ghi lại ở final_test/logs/environment.txt):

```
Python: Python 3.12.10
numpy 2.2.6
scipy 1.16.2
pandas 2.3.2
pyarrow 21.0.0
matplotlib 3.10.6
MINGW64_NT-10.0-26200 DucCuong 3.6.9-b4195d69.x86_64 2026-06-06 17:49 UTC x86_64 Msys
generated_at: 2026-08-21T05:51:45Z
```

9. Lệnh Tái lập Chính xác (Exact Reproduction Commands)

Chạy từ thư mục gốc repo, đúng theo thứ tự này. Số dòng là artifact hiện đang giữ lại, verify trực tiếp tại thời điểm build tài liệu này.

Lệnh	Đầu ra	Kết quả kỳ vọng
python -m pytest tests/test_simulator_invariants.py -q	(không có file đầu ra)	20 errors in 3.07s -- data/train.parquet chưa có trong bundle này (bị gitignore theo thiết kế, quá lớn để đóng gói); phải lấy riêng các phần chia parquet để chạy lại bộ test này cục bộ (Mục 11.3)
python train_momaql.py	data/momaql_q_table_trained.json	1.511 state (zone, hour) (không ghi đè nếu chưa chạy)

		lại toàn bộ thí nghiệm downstream)
python run_r1.py	reports/r1_validation_results.csv	25 dòng (5 policy x 5 seed)
python run_r2_ablation.py	reports/r2_ablation_raw.csv + _results.csv	15 + 3 dòng
python run_pareto_frontier.py	reports/pareto_frontier_results.csv + _summary.csv + .png	35 + 7 dòng
python run_multi_horizon.py	reports/multi_horizon_results.csv + policy_disagreement.csv	165 + 5 dòng
python run_complete_verifications.py	reports/fleet_scale_results.csv + spatial_disagreement_by_zone.csv	18 + 9 dòng
python run_spatial_candidate_pool.py	reports/spatial_candidate_pool.csv	6 dòng
python run_q_table_convergence.py	reports/q_table_convergence_daily.csv	37 dòng
python run_hypothesis1_weekly_cycle.py	reports/hypothesis1_weekly_cycle.csv	37 dòng
python run_hypothesis4_fairness_balance.py	reports/hypothesis4_fairness_balance.csv	37 dòng
python train_and_eval_mlp.py	reports/mlp_vs_tabular_results.csv + _summary.csv	15 + 3 dòng; cần torch
python make_report_figures.py	docs/*/figures/*.png (7 hình x 3 thư mục đầu ra)	sinh lại mọi hình từ các CSV ở trên
python docs/docx_report/build_research_report.py	docs/docx_report/Bao_Cao_Nghien_Cuu_FairDispatch_MOM AQL.docx	Research Report
python docs/techdoc/build_technical_documentation.py	docs/techdoc/Technical_Documentation.docx	tài liệu này

Bảng 10. *Lệnh tái lập chính xác, theo đúng thứ tự chạy.*

KHÔNG chạy `train_momaql_multipass.py` thay cho `train_momaql.py` -- đã công bố rõ là không ổn định, có thể ghi ra `data/momaql_q_table_multipass.json`, và một số script sẽ ưu tiên dùng file đó thay vì bảng canonical nếu nó tồn tại. Nếu file đó có sẵn, cách ly nó trước khi chạy bất kỳ đánh giá nào.

10. Gói Khả năng Tái lập (Reproducibility Package)

Trường	Giá trị
Git commit	3950e7fed5a59e572a3f377ffb12cc6123c3cb9e
Manifest dataset	data/train.parquet (912.375 dòng), data/val.parquet (195.508 dòng),

	data/test.parquet (195.510 dòng)
SHA-256 -- momaql_q_table_trained.json	9af13c33219f989e23a8ee9eca9e0cda3262996e34849bcc6dfab0cab5d64bdb (47,419 bytes)
Cấu hình	N_DRIVERS=200, window=60s, MAX_PICKUP_ETA=600s, lambda=0,5 mặc định (quét 0-1), gamma=0,9, alpha=0,1
Seed ngẫu nhiên	20260721, 20260722, 20260723, 20260724, 20260725 (thí nghiệm chính); 3 seed đầu dùng cho thí nghiệm cơ chế
Môi trường	Python 3.12.10, numpy 2.2.6, pandas 2.3.2, pyarrow 21.0.0, scipy 1.16.2, torch 2.7.1+cu118 (version tại thời điểm build -- kiểm tra lại bằng `pip show` trước khi trích dẫn vào báo cáo chính thức)
CPU/GPU	Intel Core i5-10300H (chỉ dùng CPU cho mọi thí nghiệm simulator/dispatch); NVIDIA GTX 1650 qua CUDA (chỉ dùng để huấn luyện MLP trong train_and_eval_mlp.py)
Số lượng test	20 errors in 3.07s -- data/train.parquet chưa có trong bundle này (bị gitignore theo thiết kế, quá lớn để đóng gói); phải lấy riêng các phần chia parquet để chạy lại bộ test này cục bộ (Mục 11.3)
Artifact kết quả	mọi file trong reports/ (16 CSV + 1 PNG + 1 manifest checksum JSON) -- Validation. Artifact Held-out Test: final_test/ (xem Mục 8).

Bảng 11. Gói khả năng tái lập (reproducibility package) đầy đủ.

reports/dataset_checksums.json là snapshot tĩnh, đã ghi từ trước; bảng ở trên tính MỚI tại thời điểm build tài liệu và là nguồn có thẩm quyền nếu 2 bên có mâu thuẫn.

11. Vấn đề Đã biết, Giả định, và Xử lý sự cố (Known Issues, Assumptions, and Troubleshooting)

11.1 Giả định (khớp Research Report Mục 11.3, theo góc nhìn kỹ thuật)

- A1. Số lượng tài xế (200) không được paper nêu rõ. Chọn theo quy mô đội xe hợp lý cho Manhattan; đổi N_DRIVERS ở đầu bất kỳ run_*.py nào để test sensitivity (xem fleet_scale_results.csv cho quét N=100/200/400 đã chạy sẵn).
- A2. Biểu diễn không gian dùng 67 zone taxi TLC chính thức, không phải đồ thị tự gộp cụm -- pickup_zone_id/dropoff_zone_id lấy trực tiếp từ cột parquet, codebase này không có bước clustering thêm.
- A3. ETA đón khách là khoảng cách haversine với tốc độ hằng số 12 mph (AVG_SPEED_MPH), không phải road routing -- đơn giản hóa có chủ đích cho simulator nhẹ (xem docstring module src/simulator.py).
- A4. Q(vùng,giờ) huấn luyện qua 1 lượt tuần tự duy nhất trên train.parquet (train_momaql.py, 1 seed=20260721) -- không phải nhiều epoch. train_momaql_multipass.py có tồn tại nhưng đã công bố rõ là không ổn định; output chẩn đoán của nó nằm ở reports/momaql_convergence.csv (5 dòng, các lượt annealed-alpha) chỉ để lưu vết, không dùng làm input đánh giá.

- A5. Không giữ lại lockfile version dependency nào; danh sách package ở Mục 3 là bộ đã audit đang hoạt động, không phải requirements.txt đã ghim version.

11.2 Vấn đề Đã biết (Known Issues)

- tests/test_simulator_invariants.py::test_no_double_booking_within_window và ::test_time_monotonicity gọi simulator với record_trace=False mặc định, nên assertion dựa trên trace của chúng hiện đang lặp trên danh sách trace rỗng. Chúng pass, nhưng không thực sự chạy qua code path phụ thuộc trace mà chúng được viết ra để kiểm tra -- chạy lại với record_trace=True cục bộ nếu cần verify đúng guarantee đó.
- run_r2_ablation.py và run_pareto_frontier.py âm thầm ưu tiên data/momaql_q_table_multipass.json hơn bảng canonical nếu file đó tồn tại trên đĩa (xem cảnh báo Mục 8) -- đây là hành vi script có sẵn từ trước, không phải bug được sửa trong đợt này, nhưng rất dễ vô tình dính phải.
- Không có structured logging framework; mỗi script chỉ print dòng tiến trình ra stdout thuần với flush=True. Capture stdout ra file nếu cần lưu log chạy lâu dài.

11.3 Xử lý sự cố (Troubleshooting)

- "FileNotFoundError: data/train.parquet" -- các phần chia parquet bị gitignore (quá to để push git bình thường); phải lấy riêng từ nơi dữ liệu của repo này ban đầu được chuẩn bị. Chỉ reports/*.csv và JSON bảng Q đã huấn luyện được track.
- Kết quả lệch nhẹ so với số trích dẫn trong Research Report -- kiểm tra reports/dataset_checksums.json (Mục 10) có khớp byte-for-byte với file data/ cục bộ của anh trước; 1 bản build parquet khác sẽ làm lệch mọi con số downstream.
- Áp lực bộ nhớ trên máy ít RAM trống -- mỗi run_*.py đã gọi gc.collect() và del object lớn giữa các seed/config; nếu vẫn OOM, giảm SEEDS về 1 giá trị duy nhất trước để cô lập xem đây là leak theo từng seed hay thật sự không đủ RAM cho working set của 1 seed.
- train_and_eval_mlp.py lỗi import torch -- đây là script duy nhất trong repo cần nó; mọi script khác chỉ dùng numpy/pandas/pyarrow/scipy thuần.